©Industrial Analytics, Dept. Civil and Industrial Engineering, Uppsala University
Course: Statistical Inference for Technological Applications (1TS325) Date: February 9, 2026

Assignment #3
Regression in Python using the Machine Learning Library PyCaret

Responsible: Mats Gustafsson

GOALS **2**
THEORY - BEFORE STARTING **4**
1. Performance metrics for regression and their abbreviations 4
2. B-fold cross-validation 5
3. Model families and associated training/fitting algorithms 6
4. User defined hyper-parameters 7
THE ASSIGNMENT - PART A and B **8**
PART A - Introduction to Regression in Python with PyCaret **9**
1. Introduction and Overview 9
2. Installing PyCaret 11
3. Overview of the Regression Module in PyCaret 12
4. Importing and Splitting a Dataset - An Industrial Example 12
5. Setting up the PyCaret Environment 14
6. Comparing All Models 16
7. Create a Model 17
8. Tune the Hyper-parameters of a Model Family 18
9. Predict using Test / Hold-out Sample for Model 20
10. Visualisation of a Chosen Model 20
11. Finalize into a Single Model/Predictions for Deployment 21
12. Predict on Unseen Data - Objective Performance Evaluation 22
13. Saving the Model 22
13. Load and Use the Saved Model 23
Questions **24**
Part B - Regression Using a Second Dataset **26**
1. Introduction 26
2. The Data Set 26
3. The Task 27
EXAMINATION and DEADLINE **29**
APPENDIX A - SOURCES **30**
APPENDIX B - LIST OF PRIMARY FUNCTIONS IN PyCaret **30**

GOALS

In this assignment, you will learn about how to solve regression problems using PyCaret, which is a popular and user friendly Machine Learning library written in Python that is intended to automate much of the Machine Learning workflow. In particular, you will learn about the following.

GENERAL:

- Regression: The problem of using a set of training examples $(\mathbf{x}_n, y_n)$ to build a prediction model $f()$ that produces predictions $f(\mathbf{x})$ where the observed response values y_n are real valued (continuous).

- Performance measures for regression problems: Metrics like mean squared error (MSE) and mean absolute error (MAE).

- Performance estimation: Cross-validation, a form of structured resampling

- Parameterized model families: Each individual model that belongs to a parameterized model family is represented by a particular set of values for hyper-parameters and the parameters that define the whole family. Example:

  Polynomial models: $f(x) = a_0 + a_1 x + a_2 x^2 + \ldots + a_Q x^Q$
  Parameters: The coefficients a_i , $i=0,1,2,\ldots,Q$
  Hyper-parameter: The degree Q of the polynomial

- Examples of parameter and hyper-parameter fitting methods designed to maximise performance on new unseen data.

- The danger/difficulty of comparing and rejecting modeling approaches without hyper-parameter tuning - which is the default approach suggested in PyCaret!!

- The importance of having an independent ("unseen") dataset at the end in order to obtain an objective (unbiased) estimate of the true performance of the final model built.

- The difficulty of choosing hyper parameters and reducing everything to a final single prediction model based on cross-validation results and/or hold-out tests sets.

PYCARET:

- The general idea of the PyCaret toolbox and why it represents an interesting tool in the field of Statistical Inference.

- Importing Data: How to import data from the PyCaret repository.

- Setting up Environment: How to set up a regression experiment in PyCaret and get started with building regression models.

- Create Model: How to create one model from a model family, perform B-fold cross-validation and evaluate regression metrics.

- Tune Model: How to automatically tune the hyper-parameters of a regression model

- Plot Model: How to analyze model performance using various plots.

- Predict Model: How to make predictions on new/unseen data.

THEORY - BEFORE STARTING

1. Performance metrics for regression and their abbreviations

Regression is about using a set of N training examples ($\mathbf{x}_n$,$\mathbf{y}_n$) to build/train a mathematical/computational function g(.) so that its predictions $\hat{\mathbf{y}}_n = g(\mathbf{x}_n)$ agree with the target responses $\mathbf{y}_n$ meaning $\hat{\mathbf{y}}_n \approx \mathbf{y}_n$, n=1,2,…,N. This is a **passive machine learning** in the sense that the whole set of N training examples ($\mathbf{x}_n$,$\mathbf{y}_n$) is already given by a supervisor, so the machine learning method used to build the model g() cannot choose any other examples.

This is in contrast to **active machine learning** where the algorithm itself sequentially is asking the supervisor for the response $\mathbf{y}_{n+1}$ correspondent to the next input $\mathbf{x}_{n+1}$ of main interest after so far have obtained the training examples ($\mathbf{x}_1$,$\mathbf{y}_1$), ($\mathbf{x}_2$,$\mathbf{y}_2$),…($\mathbf{x}_n$,$\mathbf{y}_n$). A special case of this is **a single experimental design** where all N inputs $\mathbf{x}_n$ are chosen simultaneously instead of sequentially.

Make sure you know about the following performance metrics and associated abbreviations, in the context of regression:

MAE = Mean Absolute Error
MSE = Mean Squared Error
RMSE = Root Mean Squared Error
R2 = R^2 = the coefficient of determination, pronounced "R squared"

$$MAE = \frac{1}{N}\sum_{i=1}^{N} |y_i - \hat{y}|$$

$$MSE = \frac{1}{N}\sum_{i=1}^{N} (y_i - \hat{y})^2$$

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} (y_i - \hat{y})^2}$$

$$R^2 = 1 - \frac{\sum(y_i - \hat{y})^2}{\sum(y_i - \bar{y})^2}$$

Where,
$\hat{y}$ – *predicted value of y*
$\bar{y}$ – *mean value of y*

$$\bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$$

MAPE = The mean absolute percentage error, also known as mean absolute percentage deviation (MAPD).

$$\text{MAPE} = \frac{1}{n}\sum_{t=1}^{n}\left|\frac{A_t - F_t}{A_t}\right|$$

Here A_t is the actual value and F_t is the forecast value. Their difference is divided by the actual value A_t. The absolute value of this ratio is summed for every forecasted point in time and divided by the number of data points n.

Unfortunately, the meaasure R^2 i**s often reported although it is not very informative and comes with a lot of issues**. In PyCaret, it is the default setting for ranking models according to performance… As shown from the formula it it is one minus the ratio between the sum of squared prediction errors for the prediction model and the sum of squared error for the naive (basic) model that always predict s the mean value of the response values in the training set. This means that if the prediction model is very bad, the R^2 value can actually become negative - although the name of this coefficient suggets it is a positive quantity…

In very special cases (eg ordinary least squares linear regression when the training examples x_n and y_n have zero means), the value R^2 it is equivalent to the square of the (Pearson) correlation coefficient between y_n and the predictions $\hat{y}_n$. This motivates its name R^2 but a closer look and practical examples show that can this interpretation <u>does not old in most cases</u>.

Conclusions: (1) The name R^2 gives the wrong impression that R^2 is an estimate of the correlation coefficient between y_n and the predictions $\hat{y}_n$ and therefore only has values on the interval [-1,1]. (2) R^2 is **nothing but a performance measure that compares with a naive predictor that for any input x predicts as output the mean value $\hat{y} = (y_1+y_2+\ldots+y_N)/N$. (**3) It is important to understand about R^2 and why **<u>it should not be used</u>**.

MAPE = The mean absolute percentage error, also known as mean absolute percentage deviation (MAPD).

$$\text{MAPE} = \frac{1}{n}\sum_{t=1}^{n}\left|\frac{A_t - F_t}{A_t}\right|$$

Here A_t is the actual value and F_t is the forecast value. Their difference is divided by the actual value A_t. The absolute value of this ratio is summed for every forecasted point in time and divided by the number of data points n.

2. B-fold cross-validation

Make sure you know and understand about cross-validation (CV), which is a form of structured resampling method. Below in Figure 1, B-fold CV is illustrated for the special case B=5. Keep in mind that when performing -fold CV the following holds:

- B different regression models are built using different, but partly overlapping, subsets of the original dataset D.
- The performance of each model created is estimated using a unique validation set (marked in blue in Figure 1), which has not been part of the training process at all.
- The final output is the average of the performance estimates obtained in each of the B folds.
- The cross-validation performance estimate may be viewed as an estimate of the average performance when creating models using B completely non-overlapping <u>test</u> sets (each containing ≈ N/B test examples. NOTE: For 2-fold cross-validation (B=2), there is also no

overlap also for the training sets but for B>2 the training sets are overlapping and therefore not reflecting a real world situation with non-overlapping training sets.

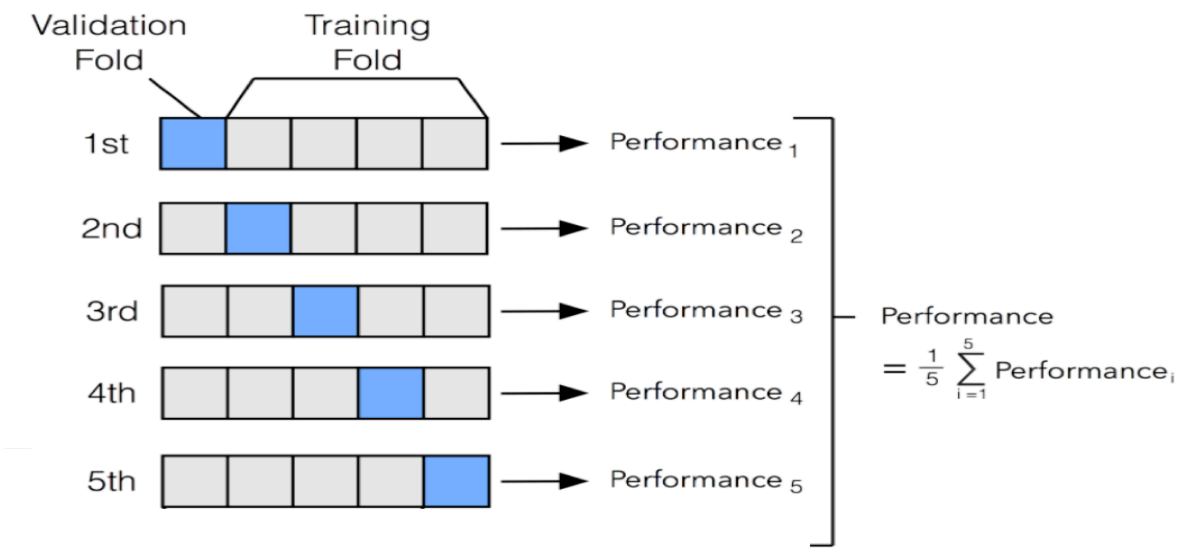

Figure 1. Illustration of the B-fold cross-validation procedure for B=5. In each fold, the performance of the model created using the training examples (gray) is estimated using the unique set of validation (hold-out) examples marked in blue. Notably, for B>2 the training sets are partly overlapping, but the test set is always independent of corresponding training set.

Remark.
B-fold cross validation may be *interpreted as a structured resampling method* where the original dataset D is used to create different subsets for training and testing. An similar approach is to instead create B bootstrap data sets D^*_b from D, and use them as training sets. Then each resulting trained model M_b is tested with the example in D that do not occur in the training set D^*_b. These test examples not occurring in the "bag" D^*_b are often referred to as the "out-of-bag" examples.

3. Model families and associated training/fitting algorithms

Make sure you know about the following types of regression models (model families) and the associated learning process:

- *Ridge Regression:* Linear Regression based on least squares fitting combined with a penalty term with penalty factor α, resulting in the predicted value $\hat{y}_n = w_o + \boldsymbol{w}^T\boldsymbol{x}_n$ for a new input $\boldsymbol{x}_n$. The function to minimize is:

$$J_{RR}(w_o,\boldsymbol{w})=(y_1-w_o-\boldsymbol{w}^T\boldsymbol{x}_1)^2+(y_2-w_o-\boldsymbol{w}^T\boldsymbol{x}_2)^2+\ldots+(y_N-w_o-\boldsymbol{w}^T\boldsymbol{x}_N)^2+\alpha||\boldsymbol{w}||^2$$

  NOTE: The penalty term $\alpha||\boldsymbol{w}||^2$ does not involve the intercept w_o. The parameter α is often called penalty factor or ridge regression (hyper)parameter.

- *K-nearest Neighbours Regression:* For each new input $\boldsymbol{x}$ of interest:

(1) Find the set

$$S_K(\boldsymbol{x}) = \{\boldsymbol{x}_{n1}, \boldsymbol{x}_{n2}, .., \boldsymbol{x}_{nK}\}$$

consisting of the K training examples that are closest (most similar) to the new input $\boldsymbol{x}$.

(2) Make the prediction

$$\hat{y}(n) = \hat{y}_n = (y_{n1} + y_{n2} + .., + y_{nK})/\mathrm{K}$$

where y_{ni} is the response of training example $\boldsymbol{x}_{ni}$.

4. User defined hyper-parameters

Make sure you know what is the user defined hyper-parameter when performing Ridge Regression and K-nearest Neighbours Regression.

Ridge Regression: The hyper-parameter here is the penalty factor α in front of the penalty term, see above. Note: The special case α =0 means ordinary least squares regression.

K-nearest Neighbours Regression: The hyper-parameter here is *K,* the number of nearest neighbours to consider when creating the prediction $\hat{y}(n) = \hat{y}_n = (y_{n1} + y_{n2} + .., + y_{nK})/\mathrm{K}$, see above.

THE ASSIGNMENT - PART A and B

The assignment here is expected to be solved by means of a Google Colab notebook written in Python. It consists of the following two parts A and B.

Part A: Go through the tutorial below and reproduce/modify the results and ideas proposed. You should also send in your code together with written answers to the associated questions. For details, se the section Examination and Deadline below.

Part B: This part will involve using the techniques you learned in Part A and applying independently to a new case. You should also send in your code and answers to the associated questions. For details, se the section Examination and Deadline below

PART A - Introduction to Regression in Python with PyCaret

1. Introduction and Overview

NOTE: ***This first section only contains an overview, so do not start coding based on this section. This will be the task in the next main section 2 below.***

PyCaret is an open-source, low-code machine learning library in Python that automates much of the Machine Learning workflow. Colab has recently (Dec 2025) changed to Python 3.12 so original PyCaret does not longer work! Therefore the following code has to be copied and run in the first Cell of your Colab notebook:

```
#RUN THIS BEFORE IMPORTING pycaret
!python3 -m pip install --upgrade pip setuptools wheel "Cython<3"
!python3 -m pip install --upgrade blinker --ignore-installed
!python3 -m pip install git+https://github.com/pycaret/pycaret.git@master --upgrade
exit() #Same as doing "Runtime -> Restart session"
```

Notably, this block of code may take some **significant (several minutes) time to complete**. Just be patient for this installation call to be completed before you continue.

PyCaret's Regression Module is a supervised Machine Learning module that is used for estimating the relationships between a response variable (often called the dependent variable, outcome variable, or target) and one or more input variables (often called independent variables, features, attributes, predictors, or covariates). Remember that the objective of regression is to predict continuous (real) values such as predicting sales, consumption, temperature, etc. Regression module provides several pre-processing features to preprocess the data.

Here follow and introduction and overview of how the PyCaret packages can be used for regression. You must keep in mind that this section is just and introduction/overview in order to give you a first overarching perspective. **Keep in mind to continue reading this section <u>without</u> trying to code! This will be the task in the next main section 2 below.**

PyCaret's regression module has many preprocessing capabilities and it comes with over 25 ready-to-use algorithms and several plots to analyze the performance of trained models. The code

```
import pycaret
from pycaret.regression import *
```

imports all the functions from the regression module of PyCaret. Here comes first a brief overview of how you might use PyCaret's regression module, more concrete details will follow later.

Setup: The first step is to set up the PyCaret regression environment using the function `setup()`. This function prepares the dataset for modeling (parameter fitting) and data transformation (pre-processing). It requires your to specify one column of your dataset to be defined as the response variable (target) for the regression task.

```
from pycaret.regression import *
reg = setup(data=your_dataset, target='your_target_column')
```

(2) Model Training and Comparison: After calling the function `setup()` as shown above, you can immediately compare different trained (fitted) regression models to find the one that seems to be most promising.

```
best_model = compare_models()
```

NOTE #1: The function `compare_models()` in PyCaret automatically uses the dataset that was initialized and preprocessed by the function `setup()` which get to know what are your data and your target column. This is a key aspect of how PyCaret is designed - it maintains the state of your entire Machine Learning workflow, starting from the function `setup()`. **In other words, everything is <u>implicitly</u> handled through <u>global variables</u> and <u>internal state management</u>, which simplifies the code you write.** However, this also means is not very transparent what is happening under the hood. This is clearly a disadvantage for users who prefer or require a more explicit approach with a full understanding of what is happening.

NOTE #2: Instead of using the implicit approach provided by PyCaret as described iabove, i**t is possible create equivalent but more transparent and explicit code as follows.**

(i) First an import is made, and then a regression experiment object is created

```
from pycaret.regression import RegressionExperiment
reg_experiment=RegressionExperiment()
```

(ii) Next, the function `setup()` is employed to defined the data to be used:

```
reg = reg_experiment.setup(data=your_dataset, target='your_target_column')
```

(iii) Finally model training and comparison is invoked:

```
best = regr_experiment.compare_models()
```

This kind of code offers greater transparency and control, particularly in complex projects where multiple experiments or configurations are being managed simultaneously. The standard approach shown under Note #1 is used by PyCaret favours simplicity. It is well-suited for quick experimentation and for users who prefer a high-level abstraction without interest in the details. The object-oriented approach described here, while more explicit and structured, might introduce additional complexity for simple task***s.*** However, this more explicit object-oriented approach is highly beneficial for complex projects as well as for beginners who wants a clear understanding and control, rather than relying on hidden implicitly handled global variables and internal state management.

(3) Model Creation: You can create a regression model using a particular combination of model family and training/fitting algorithm of your choice. For example, you can perform Ridge Regression by first making a model initiation as:

```
model = create_model('ridge')  # Ridge Regression
```

Note: Since you are not providing any value for the penalty factor α, according to the documentation, in this case the particular default value $\alpha=1$ is employed.

(4) Model Tuning: By calling the function `tune_model` you can tune the hyper-parameters of a particular model family like Ridge Regression and K-nearest neighbour. This results in a single model trained on all training examples while using the most promising hyper parameter values:

```
tuned_model = tune_model(model)
```

(5) Model Evaluation: You can evaluate the performance of the tuned (winning) model using various plots and metrics. For example

```
plot_model(tuned_model, plot = 'error')
```

(6) Model Prediction: You can use the model to make predictions on new data.

```
predictions = predict_model(tuned_model, data=new_data)
```

(7) Model Finalization: By calling the function `finalize_model`you make a final model by training on all data (training+validation) available (with the most promising hyper parameter values found before).

```
final_model = finalize_model(tuned_model)
```

(8) Saving and Loading Model: You can save your model and load it later.

```
save_model(final_model, 'my_final_model')
loaded_model = load_model('my_final_model')
```

NOTE: Each step has various options and parameters that you can customize based on your specific dataset and requirements. It's also important to properly preprocess your data before using it in PyCaret and understand the assumptions and requirements of each model you use.

2. Installing PyCaret

NOTE: ***From here and onwards, you can start to test and play with the code provided!***

Installation is achieved by running the following block of code in the first cell of your Colab notebook:

```
#RUN THIS BEFORE IMPORTING pycaret
!python3 -m pip install --upgrade pip setuptools wheel "Cython<3"
!python3 -m pip install --upgrade blinker --ignore-installed
!python3 -m pip install git+https://github.com/pycaret/pycaret.git@master --upgrade
```

exit() #Same as doing "Runtime -> Restart session"

Remember - this call may take several minutes to complete, so be patient. Then also run

```
import pycaret
from pycaret.regression import *
from pycaret.datasets import get_data
```

3. Overview of the Regression Module in PyCaret

PyCaret's Regression module `pycaret.regression` is a supervised machine learning module that is used for predicting real valued (continuous) outcomes using various techniques and algorithms. This regression module offers over 25 different model families and/or fitting algorithms. It also provides various plotting suitable for analysing the performance of models created.

4. Importing and Splitting a Dataset - An Industrial Example

Here follows an industrial example, selected to give you a concrete real world introduction to how PyCaret in particular, and Machine Learning in general, can be employed.

An insurance company wants to improve its cash flow forecasting by better predicting the patient charges using the demographic and basic patient health risk metrics at the time of hospitalization. A part of the dataset available for this task is shown below:

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520

Therefore the objective here is to fit different families of regression models to the data set available, and select the best performing regression model that predicts patient charges based on the other variables in the dataset, for a short description of them see below.

Here is a description for each dataset variable:

- **age**: age of the primary beneficiary
- **sex**: insurance contractor gender - female, male
- **bmi**: Body mass index, providing an understanding of body weights that are relatively high or low relative to height. An objective index of body weight (kg/m^2) using the ratio of height to weight, ideally 18.5 to 24.9
- **children**: Number of children covered by health insurance / Number of dependents
- **smoker**: Smoking
- **region**: the beneficiary's residential area in the US, northeast, southeast, southwest, northwest.
- **charges**: Individual medical costs billed by health insurance

The dataset for this task can be imported as follows:

```
dataset_name = 'insurance' # Replace with your desired dataset.
dataset = pycaret.datasets.get_data(dataset_name)
#check the shape of data
dataset.shape
>>> (1338, 7)
```

Thus the dataset consists of 1338 customers/patients, each described by means of 7 features. In order to demonstrate the use of the function `predict_model` on unseen data at the end, in the code below a set $D_{external}$ of 134 records **(10% of the data) is withheld from the original dataset**. One may think about this set as it consists of 134 new records, that were not available at the time when this Machine Learning task was performed. This means we have 1338-134=1204 examples for training (parameter fitting and model selection).

```
data = dataset.sample(frac=0.9, random_state=786)
data_unseen = dataset.drop(data.index) #Get all non-selected samples
data.reset_index(drop=True, inplace=True)#Reset indices of training data
data_unseen.reset_index(drop=True, inplace=True) #Reset indices in test
print('Data for Modeling: ' + str(data.shape))
print('Unseen Data For Predictions: ' + str(data_unseen.shape))

>>> Data for Modeling: (1204, 7)
>>> Unseen Data For Predictions: (134, 7)
```

NOTE: Link to dataset: https://raw.githubusercontent.com/pycaret/pycaret/master/datasets/insurance.csv

5. Setting up the PyCaret Environment

The PyCaret function `setup()` initializes the environment in PyCaret and creates the transformation pipeline to prepare the data for subsequent modelling and deployment, and automatically splits the data into one for training and for testing (with proportions 70% and 30% as default). Example:

```
from pycaret.regression import *
s = setup(data = data, target = 'charges', train_size=0.7, session_id=123)
```

This function must be called before executing any other function in PyCaret. In addition to non-mandatory inputs *it must have the following two mandatory inputs: a pandas dataframe containing the data available for both training and independent hold-out testing, and the name of the target column you want to predict.*

Once the function call using `setup()` has been successfully completed, it displays an information table/grid which contains several important pieces of information. Most of the information is related to the pre-processing pipeline which is constructed when `setup()` is executed. The majority of these features are out of scope for the purposes of this tutorial. Above the parameter `train_size` is set to the default value 0.7, which means that the call `setup()` automatically splits the data set called `data`into 70% for training and 30% for testing.

All other input parameters are optional and are used to customize the pre-processing pipeline. When the function `setup()` is executed, PyCaret will automatically choose the data types for all features based on certain properties. The data type should be chosen correctly, but this is not always the case. Therefore PyCaret displays a table/grid containing the features and the corresponding chosen data types. If all of the data types are correctly chosen, enter can be pressed to continue or quit can be typed to truncate the current computational experiment. Ensuring that the data types are correct is really important in PyCaret as it automatically performs multiple type-specific preprocessing tasks which may have great impact on the final Machine Learning models obtained.

A few important things related to the table/grid are:

session_id: A number used as a random number generator seed in all functions for later reproducibility. If no session_id is passed, a random number is automatically generated that is distributed to all functions. In this experiment, the session_id is set as 123 for later reproducibility.

Missing Values: When there are missing values in the original data, this will show as True. For this experiment, there are no missing values in the dataset.

Numeric Features: Number of features found to be numeric. In this dataset, 3 out of 7 features are found to be numeric. From the table above, it should be clear that the categorical ones are sex, smoker and region.

Categorical Features: The number of features found to be categorical. In this dataset, 3 out of 7 features are found to be categorical. From the table above, it should be clear that the categorical ones are sex, smoker and region.

Transformed Train Set: Displays the shape of the "transformed" training set D_{train}. Notice that the original shape of (1204, 7) is transformed into shape (842, 10). Here two key changes have occurred:

(1) Due to the default split **70% for training and 30% for testing**, a training set D_{train} of size 842 (70% of 1204) samples have now been extracted from the the original set (obtained after removing the 134 external examples before staring the actual data analysis and modeling).

(2) The number of features has increased from the original 7 to the current 10. This is because of "categorical encoding", which means that the categorial variables are encoded numerically by binary numbers/vectors. For simplicity assume that the non-categorical variables are denoted x_1, x_2, x_3 and x_4. Then gender can be encoded as one binary number x_5 which equals $x_5=0$ if the patient is female and $x_5=1$ otherwise. Similarly, the status of being a smoker or not can be encoded by one binary variable x_6. Finally the four possible regions can be encoded using so called "one-hot-encoding" where for example $x_7=1, x_8=0, x_9=0, x_{10}=0$ corresponds to south, $x_7=0, x_8=1, x_9=0, x_{10}=0$ to east, $x_7=0, x_8=0, x_9=1, x_{10}=0$ to west and $x_7=0, x_8=0, x_9=0, x_{10}=1$ to north.

Transformed Test Set: Displays the shape of the transformed test/hold-out set D_{test}. There are 362 samples in the test/hold-out set. This split is based on the already mentioned default split 70% +30%. These proportions can be changed using the parameter `train_size` in the `setup()`.

Linear Regression
Lasso Regression
Ridge Regression - 'ridge'
Elastic Net
Orthogonal Matching Pursuit Bayesian
Ridge Gradient Boosting Regressor
Extreme Gradient Boosting
Random Forest Regressor
Decision Tree Regressor
CatBoost Regressor
Light Gradient Boosting Machine
Extra Trees Regressor
AdaBoost Regressor
K Neighbors Regressor - 'knn'
Lasso Least Angle Regression
Huber Regressor
Passive Aggressive Regressor
Least Angle Regression

Figure: All models in PyCaret

6. Comparing All Models

Comparing all models (modeling approaches) by evaluating their performances is the recommended starting point in PyCaret for modeling, once the function call using `setup()` is completed. **Calling the function `compare_models()` will train/fit all model families in the model library using conventional B-fold cross-validation** based on the examples in the training set D_{train}. If not specified by the used, the value of B is set to B=10.

Calling `compare_models()` will also crank the fitted model families based on their performance metrics. A list of the model families (modeling approaches) in the library are presented above, where the two indicated in bold (Ridge Regression and K-nearest neighbour regression) are chosen for illustration in this assignment. Also shown are the corresponding abbreviations used by PyCaret:

NOTE: Several methods here offer linear prediction models. However, they are using different algorithms and hyper parameters for the parameter fitting: Linear Regression, Lasso Regression, Ridge Regression, Bayesian Ridge Regression, Least Angle Regression, and Elastic Net.

The function `compare_models()` prints out results that show performance measures like average MAE, MSE, RMSE, R2, and MAPE across the B folds along with the training time each model needed.

```
compare_models()
```

Using only a single line of code, we have trained and evaluated all the above modeling approaches using cross-validation. The scoring results printed above highlights the models with the highest performing metric for comparison. By default the sorting is based on the default metric R2 (highest to lowest), which can be changed by passing a parameter called `sort`. For example, the call

```
compare_models(sort = 'RMSE')
```

will sort the grid by RMSE (lower to higher since lower is better).

If you want to change number of folds in the cross-validation from the default value B=10 to some different value, then you can use the parameter `fold`. For example the call

```
compare_models(fold = 5, sort = 'RMSE')
```

will compare all models using a 5-fold cross-validation instead. Reducing the number of folds will reduce the training time. Some other important user defined parameters available for the function `compare_models()` are:

`n_select:` In addition to the full list of comparisons, it also return the names of the `n_select` top models based on the performance metric selected like RMSE.

`sort:` allows you to sort results by your favourite metric, for example MAE.

`include:` if you only want to use a certain subset of models in your comparison

`exclude:` if you want to exclude a certain subset some models from your analysis

Here follows two code snippet examples showing of how to apply some of these parameters:

```
# Example 1
# compare all models according to the metric MAE, show
# table with all results, and then print a list of the
# top 3 models according to the cross-validation
# based performance estimation.
ranked_models=compare_models(n_select = 3, sort = 'MAE')


# Example 2
# compare only ridge regression and knn, sort using RMSE
# 'ridge'= ridge regression
# 'knn'  = k-nearest neighbour
# regression)
ranked_models=compare_models(include = ['ridge', 'knn'], sort = 'RMSE')
```

7. Create a Model

The function `create_model` is a key function in PyCaret and it is often the foundation behind many PyCaret functionalities. As the name suggests this function trains and evaluates a model using cross-validation (specified in terms of the fold parameter). The output prints a scoring grid that shows performance metrics like MAE, MSE, RMSE, R2, and MAPE for each fold.

For the remaining part of this assignment, we will work with the following modelling strategies:

```
Ridge Regression ('ridge')
K-Nearest Neighbors Regressor ('knn')
```

Notably, the selections are **for illustration purposes only** and does not at all mean these two are the top-performing for this type of data (in fact they are not). As indicated already above, there are 25 regression modelling approaches available in the model library of PyCaret. To see a complete list just write models(). Here follows the code required to create a Ridge Regression model and a K-nearest neighbour model.

7.1 Ridge Regression

```
ridge = create_model('ridge')  # alpha=1 is the default
print(ridge)
```

7.2 K-Neighbors Regressor

```
knn = create_model('knn') #k=5 is the default used here
print(knn)
```

Notice that the mean score of all models agrees with the score printed in `compare_models`. This is as expected, because the output results from `compare_models` are also the average scores across all B cross-validation folds. As for `compare_models`, if you want to change the number

of folds B from the default value of $B=10$ to a different value using `create_model`, then you can use the fold parameter in the function `create_model`. For Example:

```
create_model('ridge', fold = 5)
```

will result in running Ridge Regression using 5-fold cross-validation.

8. Tune the Hyper-parameters of a Model Family

When a model is created using the function `create_model`, it uses the default values of the hyper-parameters to train the model. For example, Ridge Regression is performed with the penalty factor chosen as $\alpha=1$, and K-nearest neighbour regression is performed with $K=5$ as the default value.

In order to tune hyper-parameters, the function `tune_model` is used. This function automatically tunes the hyper-parameters of a model using a grid search by means of the function `RandomGridSearch` on a pre-defined search grid/space. Again cross-validations is used to estimate the performance for the different values of the hyper-parameters specified by the search grid. The output prints results showing performance metrics like MAE, MSE, RMSE, R2, and MAPE by fold. To use a personal (tailor made) search grid, you can pass a parameter `custom_grid` in the call to the function `tune_model`. Here follows an example where the search grid is defined by the variable `ridge_grid` below:

```
# define tuning grid
ridge_grid = {'alpha' : [0, 0.2, 0.4, 0.6, 0.8, 1, 2]}
# tune model with custom grid and metric = MAE
tuned_ridge = tune_model(ridge, custom_grid = ridge_grid, optimize = 'MAE')
```

Thus the function `tune_model` tunes/selects the hyper-parameters of the model. The output of this function contains the results of the cross-validation method applied per fold. The output also contains a tuned single model which has been trained on the entire set of training examples D_{train} using the most promising value of the hyper-parameter found on the search grid. Thus, after the best hyper-parameter has been identified using B-fold cross-validation, `tune_model` retrains the model using this hyper-parameter on the entire training dataset D_{train}.

8.1 Ridge Regression

```
tuned_ridge = tune_model(ridge)
print(tuned_ridge)
```

8.2 K-Neighbors Regressor

```
tuned_knn = tune_model(knn)
print(tuned_knn)
```

By default, `tune_model` optimizes R^2 but this can be changed using the parameter `optimize`. For example, the call `tune_model(ridge, optimize = 'MAE')` will search for the hyper-parameter α of the Ridge Regression model that results in the lowest MAE instead of the highest R^2. The pros and cons of using different performance metrics is beyond the scope of this tutorial.

After performing B-fold cross validation using `tune_model` above twice, one for ridge and one for kNN, PyCaret automatically builds two final models using all examples in D_{train}, each of them built with the most promising hyper parameter value found in the corresponding cross-validation. In this case, it found that the weights in Ridge Regression should be fitted when using a particular value α^* for the penalty parameter. Similarly, PyCaret found that the number of neighbours should have a particular value K^* in order to perform well on independent hold-out examples. *Thus a tuned Ridge Regression model was built using α^* based on all examples in the training set D_{train} and a corresponding K-nearest neighbours regression model was built using the same data when using the hyper-parameter value K^*.*

This automated approach used in PyCaret comes with a nontrivial risk because the selected hyper parameter values are not necessarily the best when training with all data in D_{train}. For example, if using B=5 folds, then the size of the B different training sets have been 80% of all data in D_{train}. However, **other hyper parameters might have been much better to use when all the available data are used for training**. Thus there are two important problems here:

(1) The cross-validation performance estimate does not correspond to the case when all available data in D_{train} are used for training. Therefore the final training using all data in D_{train} after hyper-parameter selection may give much better/worse results than the ones predicted based on cross-validation.
(2) Selecting model between ridge regression and K-means regression based on the cross-validation results may be misleading because of (1).

In other words, it is not obvious that the most promising modeling approach found using B-fold cross validation will be the best when training with all examples in D_{train}: If we have to choose between the Ridge Regression model or K-nearest neighbour regression model from above, it seems natural to choose the one that provides the best average performance in the B-fold cross-validations procedures performed above. However, **none of these two final models have been evaluated on any independent test examples, so it is no longer obvious/certain which of the two is most promising!** Perhaps other values of the hyper-parameters could have been used to obtain better performance for both methods.

For example, the "learning curve" (performance as a function of the size of the training set) for the K-nearest neighbour approach might have a steeper slope than for Ridge Regression. This would mean that just a slight increase of the number of training examples would result in substantial improvements in performance for the K-nearest neighbour method compared to Ridge Regression.

In order to make better understand which of the two models actually performs better when training with all examples in D_{train}, these tuned models must be tested on new independent examples… This can and should be done using the already created test set D_{test}. Only after

estimation the performance using this set D_{test}, it will be possible to decide which of the two models is most promising and therefore the one to keep for final use/production.

Remark. An alternative approach not considered in PyCaret is to skip retraining using all data in $\underline{D_{train}}$. Instead the B models obtained in the B-fold cross validation could have been kept for the most promising modeling approach. Then predictions would be made as the average of the predictions produced by each of these B models. More about this below.

9. Predict using Test / Hold-out Sample for Model

As already explained, in the insurance related case study considered above, 30% (362 samples) of the original data set were separated out to become the independent test set D_{test}. These examples can now be used to test the Ridge Regression and the K-means models built using all data D_{train} when employing the most promising hyper parameter values (α^* and K^* respectively) - according to the B-fold cross-validation procedure employed previously.

A first sanity check is to evaluate if there are substantial differences in the performance estimates obtained using the cross-validation and using the independent test set D_{test}. Again keep in mind that the models used in the B-fold cross validation were not trained using all data while the final model is. Consider the results after calling

```
predict_model(tuned_ridge)
```

The R^2 on the independent test set D_{test} is 0.71 in this case, which should be compared to 0.75 obtained using cross-validation above. If there is a large deviation between the result obtained here and the cross-validation result from above, this suggests that the hyper parameters selected from the cross validation are not suitable when training with all data.

Selection of model: The tuned model which is performing the very best on $D_{test,}$ according to the metric RMSE, should be selected as your final model.

10. Visualisation of a Chosen Model

The visualisation function `plot_model` can be used to analyze the performance across different aspects such as Prediction Error and Feature Importance. This function `plot_model` takes a trained model object and returns a plot based on the test/hold-out set. There are over 10 plots available, see the documentation of `plot_model` for a list of them. Here follows some examples.

10.1 Prediction Error Plot

```
plot_model(tuned_ridge, plot = 'error')
```

10.2 Feature Importance Plot

```
plot_model(tuned_ridge, plot='feature')
```

11. Finalize into a Single Model/Predictions for Deployment

The Machine Learning workflow recommended in PyCaret starts with the setup, followed by comparing all modeling approaches using the call to `compare_models` and shortlisting a few candidate approaches (based on the metric of interest) to perform several subsequent hyper-parameter tuning attempts. After a final model selection based on prediction results using the test set D_{test}, the most promising combination of model family and fitting algorithm has been identified.

At this point, PyCaret recommends that one builds a final model using this most promising combination (of model family and hyper parameters) applied to all data available, meaning all examples in both D_{train} and D_{test}. This is what is provided by the PyCaret function call

```
final_ridge = finalize_model(tuned_ridge)
```

Strengh: The obvious strength of this strategy is that significantly more training examples are used to create the desired regression model. With more training examples, the performance is expected to become better.

Weakness/dangers: An obvious weakness/danger of this strategy is again that **the model family and the hyper-parameter value(s) were not selected based on such a relatively big dataset that currently is used for parameter fitting.** This means that the final model produced here might be based on a suboptimal model family and/or on a suboptimal hyper-parameter values(s). Moreover, using this strategy there is no longer any independent data left out for a final unbiased performance evaluation (unless you have put aside external data like our set $D_{external}$ created in the very beginning). **If not available, we can only cross our fingers hoping/believing that the resulting regression model will perform well on new independent data…**

Based on the above analysis regarding strengths and weaknesses, there are at least three alternatives.

Alternative I: Keep the most promising tuned model selected based on its best performance on D_{test}. Since this model has performed well on the independent test set D_{test}, we hope it will perform well when applied to other new data and no retraining takes place. Assuming Ridge Regression has been selected as the most promising method, we simply make the assignment

```
final_ridge = tuned_ridge
```

Alternative II - suggested in PyCaret: Assuming Ridge Regression has been selected as the most promising method, we will retrain the with all data available (for the selected most promising hyper-parameter obtained earlier) available in D_{train} and D_{test} . This is done using the call

```
final_ridge = finalize_model(tuned_ridge)
```

Notably, this final model has never been tested on any independent test data, so its performance is unknown and uncertain. However, it is hoped that the model performs well because its hyper-parameter has been carefully tuned before - although using smaller training sets.

Alternative III - "ensemble" model: Assuming Ridge Regression has been selected as the most promising method, in this alternative we keep all B models build in the cross validation using the best performing hyper parameters. Then predictions for new examples are determined as the mean value of the B predictions these models produce individually. A great advantage here is that the examples in D_{test} are still unused. Therefore they can be used to obtain an unbiased estimate of the true performance of this "ensemble" model. This Alternative III will NOT be explored in this assignment as PyCaret does not provide any user friendly support for this case.

12. Predict on Unseen Data - Objective Performance Evaluation

After producing the very final model above, using Alternative I or II (Alternative III is not considered more in this assignment), a final sanity check should be performed by testing the model on unseen data that have not been involved in any part of the model development and model selection.

As already seen above, the function `predict_model` can be used to make predictions for any unseen/new dataset. Therefore we will now make predictions using the examples in the set $D_{external}$, which is labelled `data_unseen` in the Python code. This is the set we created in the beginning of this assignment - it contains 10% (134 samples) of the original dataset that has never been exposed to the PyCaret toolbox. The following call does the job:

```
unseen_predictions=predict_model(final_ridge, data=data_unseen)
unseen_predictions.head()
```

This means the column `prediction_label` is added to the set `data_unseen`. This column contains the predicted values using the model `final_ridge`.

Since the data used here for performance estimates are completely independent of the modeling development and final model selection, they can be used as to obtain an unbiased estimate of the expected performance for the case when predicting using completely new data. It is a sanity check in the sense that if **the performance here is not sufficiently good, then this final model must not be employed. In this case, the whole modeling process followed should be terminated until more data have been collected that can give better performance and more reliable performance estimates.**

13. Saving the Model

We have now finished the experiment by obtaining the model `final_ridge`. This brings us to the end of our experiment. However one question is still to be asked: What happens when you have more new data to predict? PyCaret's built-in function `save_model` allows you to save the model along with the entire transformation pipeline for later use:

```
save_model(final_ridge,'Final Ridge Regression Model')
```

13. Load and Use the Saved Model

To load a saved model at a future date in the same or an alternative environment, we would use PyCaret's function `load_model` and then just apply the saved model on new unseen data for prediction:

```
saved_final_ridge = load_model('Final Ridge Regression Model')
```

Once the model is loaded into the environment, you can simply use it to predict any new data using the imported function `predict_model`. Below it is shown how one can use the the loaded model to make prediction for new examples stored in the variable `data_new`:

```
new_prediction = predict_model(saved_final_ridge, data=data_new)
new_prediction.head()
```

Questions

1. Why B-fold cross-validation: Using B-fold cross-validation, each model built (fitted) is tested on "validation" (hold-out) examples that were not used to build the model, although still part of the whole set of training set D_{train}. This results in B performance estimates. *Explain the fact that the average of these B performance estimates is a reasonable estimate of the expected performance when employing the current model family using training sets of the same size as the B subsets D_{train} of used here.* Hint: The B-fold cross validation approximates/simulates the situation where one is training models using $\approx(1-1/B)N$ training examples and then test them using $\approx N/K$ test examples.

2. The hyper-parameter for a Ridge Regression: The hyper-parameter for a Ridge Regression is often called α or λ. *What does this hyper-parameter control?* Explain by discussing what happens with the model coefficients $\boldsymbol{w}$ of the associated linear model $y=w_0+\boldsymbol{w}^T\boldsymbol{x}$ in the two extreme cases when this parameter has the value is zero and when it is very large.

3. The hyper-parameter for the K-nearest neighbour regression: The hyper-parameter for the K-nearest neighbour regression model considered here is denoted K. (a) What is it that this hyper-parameter K is specifying? (b) Explain why the prediction value will be the same (constant) for all inputs $\boldsymbol{x}$ when $K=N$ (meaning the hyper-parameter K is set to be equal to the number of data points N).

4. *Compare performance metrics after final parameter tuning:* (a) Consider the tuned Ridge Regression and the tuned K-NN regression models in terms of their Root Mean Squared Errors (RMSEs) on the independent test set D_{test}. Which of the two performs best? (b) How do the resulting performance estimates obtained using D_{test} compare with the performance estimates obtained from the previous cross-validation procedure (used to tune the hyper-parameters)? What are the reason for expecting better performance ? What are there reasons for expecting worse performance? Hint: We do not know anything about what are the optimal hyper parameter values when training with all data in D_{train}.

5. *Pros and cons when choosing the number of folds B in B-fold cross validation:* The parameter 'fold' in the function `compare_models()`? defines the number of folds B used in the B-fold cross-validation procedure employed. Remind yourself about how cross-validation works and discuss the pros and cons of making the parameter 'fold' bigger/smaller in terms of
   (a) computational effort
   (b) similarity (overlap) between the training sets.

   Consider in particular the special case "leave-one-out" cross-validation $B=N$, where only one example is left out for testing in each fold. Also consider the special case $B=2$.

6. *Why avoiding the use of `compare_models`:* The function `compare_models` is recommended by PyCaret to be used in a first step to compare a panel of different model families/algorithms by means of cross-validation in order to find the most promising ones. Then the most promising approaches are selected and their hyper-parameters are optimized in a

second step using the function `tune_model`. However, this strategy can be very misleading, resulting in a suboptimal final model.

(a) Explain why this strategy may lead to a suboptimal final model by means of the following hypothetical but illustrative example: Assume that when using the hyper-parameter value $K=2$ with K-nearest neighbour regression, the performance is much better than what can be achieved using Ridge Regression for any choice of its hyper-parameter α. Also assume that for the default value $K=5$, the K-nearest neighbour regression approach performs worse than Ridge Regression using the default value $\alpha=1$ for its penalty factor. Now consider what happens after employing `compare_models`, where only the the most promising modeling approach (in this case Ridge Regression) will be selected to be tuned using function `tune_model`.

(b) Based on your answer in (a), explain why one should avoid using `compare_models` and instead directly use `tune_model` for the comparison of different modeling approaches.

7. *Why avoiding the use of the function `finalize_model`:* PyCaret suggests that after selecting the most promising combination of hyper-parameter value(s) and conventional model parameters, a final model should be built using all data available by means of the function `tune_model`,. The the call

```
final_ridge = finalize_model(tuned_ridge)
```

would result in a model `final_ridge` where all training examples in D_{train} and D_{test} have been used for the parameter fitting. Thus the already selected value for the hyper-parameter will not be tuned here, only the conventional parameters will be fitted to the data. Discuss the following two drawbacks of this idea to use all data available to create a final model:

(a) The already performed hyper-parameter selection was based on training sets during cross-validation. They are non-overlapping subsets of D_{train} and therefore smaller than the total number of examples available for training when now merging D_{train} and D_{test}.

(b) No independent data are available to estimate the performance of this final model produced (unless a set of "unseen" data is collected afterwards or is created before the modeling procedure started - like our $D_{external}$).

Part B - Regression Using a Second Dataset

1. Introduction

In this part B, you will apply what you have learned in part A to a second dataset.

Buildings are responsible for over 40% of total energy consumption in the EU. A large share of this energy comes from a building's heating and cooling requirements. Modelling a building's energy efficiency and being able to predict the heating and cooling loads of a building based on its physical properties is a task of great importance to urban and city planners. In this assignment, you will develop a model to help predict a building's heating and cooling loads.

2. The Data Set

The following data is based on 768 various building shapes with different traits and properties. The dataset consists of these 768 samples and 8 different features. Urban planners would like your help in modelling the energy performance of these buildings. Specifically, their heating and cooling loads. This will allow them to make better decisions with regards to energy efficiency.

The dataset you need is built into PyCaret. After importing PyCaret you can load the dataset directly with the following command:

```
from pycaret.datasets import get_data
dataset = get_data('energy')
```

Here below is a visualisation of the first part of the dataset (table):

	Relative Compactness	Surface Area	Wall Area	Roof Area	Overall Height	Orientation	Glazing Area	Glazing Area Distribution	Heating Load	Cooling Load
0	0.98	514.5	294.0	110.25	7.0	2	0.0	0	15.55	21.33
1	0.98	514.5	294.0	110.25	7.0	3	0.0	0	15.55	21.33
2	0.98	514.5	294.0	110.25	7.0	4	0.0	0	15.55	21.33
3	0.98	514.5	294.0	110.25	7.0	5	0.0	0	15.55	21.33
4	0.90	563.5	318.5	122.50	7.0	2	0.0	0	20.84	28.28

In the following table you will find short description of the 8 variables that this dataset contains.

Variable	Description
Relative Compactness	Ratio of surface area to volume
Surface Area	Surface area of building (m^2)
Wall Area	Wall area of building (m^2)
Roof Area	Roof area of building (m^2)
Overall Height	Height of building (m)
Orientation	Building's orientation - 2: North, 3: East, 4:South, 5: West
Glazing Area	Total area of the wall which is glass - 0%,10%,25%,40%
Glazing Area Distribution	Distribution of glazing area within the whole building - 1:Uniform, 2:North, 3:East, 4:South, 5: West Where uniform has 25% glazing on each side, and all the remaining categories have 55% in the corresponding direction with 15% on the remaining sides
Heating Load	Amount of energy required to maintain an interior temperature that is suitable to the building's inhabitants (kWh/m^2)
Cooling Load	Same as above except that the energy is for removing heat rather than adding it (kWh/m^2)

NOTE: You do **not** need to have a deep and technical understanding of all these variables. The explanations in the table above are provided to give you some intuition concerning the task at hand.

However, the basic idea should be clear to you: The task, which is described in more detail below, is to predict the heating load for a new previously unseen building - based on a set of features that characterise that particular building.

3. The Task

Your task here is to follow the following steps and address the following questions:

1. Start by separating out 10% of the data as "unseen data" to be evaluated at the very end. We will "pretend" this is data the model has not seen at all before and use it as a "real-life" scenario.

2. Set up the environment with heating load as the target (outcome) variable of interest to predict. Compare the models obtained when employing ordinary least squares regression, default ridge

regression, and default K-nearest neighbours regression as in part A. Which is the best model according to the RMSE? Which is the best model according to the performance metric R^2?

3. Let's try create Ridge Regression and K-neighbours regression models by tuning their hyper-parameters.

   (a) For the Ridge Regression model, create a custom tuning grid with the following values of the penalty coefficient: {0, 0.1, 0.3, 0.5, 0.7, 1, 2, 4, 8}.

   (b) For the K-nearest neighbour regression, create a custom grid with the following values of K: {3,5,7,9,11}

   (c) Report the mean RMSE values for the different hyper-parameter values used, or at least the best RMSE found. Does any hyper-parameter setting result in better mean RMSE than what was obtained when using the default settings in step 2?

4. Using your most promising Ridge Regression model found so far, determine the two most important variables for this current task of predicting the heating load. **Remark.** Do not try the same with your best K-neighbours regression model because PyCaret will give an error!

EXAMINATION and DEADLINE

The examination of this exercise will be in the form of:

1. Presenting and discussing your results orally during one already scheduled mandatory seminar. **Your notebook and your pdf** should be uploaded to the course home page **before the deadline, for details see course homepage**. Make sure that all files you are submitting have your first name and surname as part of the file name, for example like:

```
Assignment_3_reflections_Mats_Gustafsson.pdf
```

During this mandatory seminar, you need to have access to a computer with camera and your full name visible so that you are able to share your results with others in the seminar via a course specific online meeting. The permanent link to this online meeting is available on the course home page. If you are not able to attend it, contact the main teacher in order to book a separate seminar with him/her.

2. Questions about these exercises and the associated theory and Python code during the final written exam.

Enjoy and Good Luck!

APPENDIX A - SOURCES

https://towardsdatascience.com/introduction-to-regression-in-python-with-pycaret-d6150b540fc4

https://colab.research.google.com/github/pycaret/pycaret/blob/master/tutorials/Tutorial%20-%20Regression.ipynb

https://towardsdatascience.com/every-data-scientist-should-use-pycaret-45a1e8e984be

https://pycaret.gitbook.io/docs/

https://pycaret.gitbook.io/docs/learn-pycaret/videos

APPENDIX B - LIST OF PRIMARY FUNCTIONS IN PyCaret

The PyCaret regression module is a part of the PyCaret Machine Learning library and offers a variety of functions to streamline the process of performing regression analysis. Here follows a list of the primary functions available in this module. These functions provide a comprehensive toolkit for handling various aspects of regression analysis, from data preparation and model selection to evaluation and deployment. It's worth noting that PyCaret is continuously updated, so it's a good idea to refer to the official PyCaret documentation for the most current information and additional details on these functions.

setup: This function initializes the environment in PyCaret and sets up the data for modeling.

create_model: It's used for creating a model. It takes the ID of a model as input and returns a trained model object.

compare_models: This function compares different models and scores them using cross-validation. It helps in identifying the best model for your data.

tune_model: It's used for hyper-parameter tuning of a given model to improve its performance.

ensemble_model: This function is used for “ensembling” a given model. It can be used for bagging, boosting, and creating a blend of different models.

blend_models: Used to blend multiple different models to create an ensemble. It averages the predictions of all models**.**

stack_models: This function stacks multiple models for “ensembling”.

plot_model: It's used for plotting different aspects of a given model, like feature importance, error plots, etc**.**

evaluate_model: This function displays a user interface for all of the available plots for a given model.

interpret_model: Used for model interpretation and explaining predictions, especially useful for complex models.

predict_model: This function is used to make predictions on new data using a trained model.

finalize_model: This function finalizes the model for deployment. It trains the model on the entire dataset including the holdout set.

save_model: It's used to save the model along with its entire transformation pipeline.

load_model: This function is used to load a saved model.

automl: It returns the best model out of all models created in the current environment based on a certain metric.

pull: Used to pull the latest experiment table into a pandas DataFrame.

models: This function returns a table of models available in the model library.

get_config: It's used to retrieve the global variables created as a result of the setup() function.

set_config: This function is used to reset the global variables.

get_logs: It's used for retrieving the logs from the experiment.